



The Shed, explained

A file and folder browser on the starter you sent, built the way I build my own project: shape and tests first, every decision written down, and a live site.

MAPLARGE DEVELOPER TEST PROJECT | STEVEN STOUT | 29 SEPTEMBER 2026

LIVE theshed.stevenstout.biz

CODE github.com/SteveStout/TheYard, [samples/maplarge](#)

RESUME AND ABOUT on the site, one click each

What it is

One configurable home directory, one API, one page. The address bar is the whole state.

An ASP.NET Core 10 API over a home directory set by configuration, and a page in TypeScript with no framework. It browses, searches by name or wildcard, uploads with a progress bar, downloads, makes folders, moves, copies and deletes, inside a native dialog with one trigger. Every failure is a sentence the page shows. Twelve decision records are served from the running app, each with the code it decided about read from the build as you read it.

QUICK START

```
dotnet run          # localhost:5120, browsing sample-home/
dotnet test         # 112 xUnit tests, warnings as errors
npm install && npm test # 21 node tests over the compiled modules
```

OR SKIP THE BUILD, THREE LINKS

- theshed.stevenstout.biz/?path=reports/2026: the dialog, open in a folder.
- [/?q=*.csv&sort=size&dir=desc](https://theshed.stevenstout.biz/?q=*.csv&sort=size&dir=desc): a search, sorted, as a link.
- [/?view=docs&doc=adr-003-the-line-a-path-cannot-cross](https://theshed.stevenstout.biz/?view=docs&doc=adr-003-the-line-a-path-cannot-cross): the security record, its code live. Read ADR-003, 005 and 009 first; the README is the map.

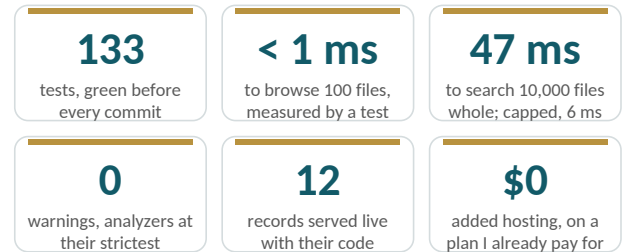
The brief, and where each item is

You asked for	Where it is
Builds in VS, Rider, VS Code, CLI	TestProject.sln, your file kept by name, on .NET 10; the app references no package.
JSON API: browse and search	FilesController; snake_case; one RFC 9457 problem document on every failure, with a trace id.
Home directory by variable	Files:Home or FILES__HOME. HomePath is the one class that turns a request path into a real one.
Deep-linkable state in the URL	Six query keys, one parser and serializer, one navigate(). Back, reload and a pasted link take the same path.
Vanilla SPA, client rendered	TypeScript compiled by tsc alone; src/lib is pure, src/ui renders; no innerHTML anywhere, and a test proves it.
Upload and download	Multipart with a bar per file, 409 answered in place; downloads stream with range requests.
Counts and sizes per view	Totals on every reply, computed on the server for exactly what is listed, plus took_ms.
Bonus: dialog, writes, performance	A native dialog and one button; delete, move and copy ask in the row; a 10,000-file measurement.

How it was built and shipped

- **Green before every commit.** 133 tests, warnings as errors, and a headless Chrome pass of the running page (deep links, search, Back, upload, delete, Escape) with zero console errors.
- **Live at no added cost.** A third web app on the App Service plan my project runs on, its own free managed certificate, the image in the same registry, rolled by a workflow whose identity holds Website Contributor on that one site and nothing wider. The Bicep and the workflow are in the folder.
- **Built with AI, and written down (ADR-011).** I decided the shape and the test list first, Claude drafted against them, the build ran on my machine, and I read everything before it shipped. My first push had a folder-name case collision Windows hides and Linux does not; the container found it, and the link test now checks case exactly.

By the numbers



Over 10,000 files on the build machine (ADR-008)

lower is better



Six choices, and why

- 1 Your starter, kept**
Your names, your controller shape, your one page. Three lines added to the project file (ADR-001).
- 2 Dependencies inward**
One project, four folders. A test reads every using and fails on one that points the wrong way (ADR-002).
- 3 One pure security class**
Three string refusals before any filesystem touch, then containment; the same tests on both OSes (ADR-003).
- 4 The address is the state**
Nothing is kept anywhere else. The first keystroke of a search pushes; the rest replace (ADR-005).
- 5 Measured, not claimed**
Search is lazy, stops at a cap and says so; a test prints the timings on every run (ADR-008).
- 6 Rules beside their tests**
A table of every standing rule and the test that holds it; a test reads the table (ADR-009).

Taking it further: my larger project is the worked example

TheYard (theyard.stevenstout.biz) is the same practice at full size: .NET 10 and React, Azure SQL and Cosmos DB, 86 records, about 1,940 test runs per gate, two sites on one plan. Each step below is something the Yard already does.

Next on the Shed	How the Yard already does it
A committed browser test	142 Playwright specs in the gate, located by role and name; axe holds nine views to WCAG 2.1 AA.
Paging behind the search cap	The inventory API pages 100,000 vehicles behind Load more; the page never asks for the whole set.
Logs that outlive the container	Every request and error kept in Cosmos DB for three years, read on the Admin tab (ADR-071, 073).
The path guard as a Files card	The Admin tab is already a product: health, machines, timing, kept logs, each a card (ADR-080).

The line a path cannot cross (ADR-003)

After the empty and rooted cases: two refusals as strings, then the joined path must still start inside the root. Pure, so the same tests run on Windows and Linux.

```
Domain/HomePath.cs, Resolve
string[] segments = trimmed.Split(Separators,
    StringSplitOptions.RemoveEmptyEntries);
foreach (string segment in segments)
{
    if (segment is "." or "..")
    {
        throw new
            PathRefusedException("A path cannot contain '.' or '..'
            segments.");
    }
    if (segment.IndexOfAny(System.IO.Path.GetInvalidFileNameChars()) >=
        0)
    {
        throw new
            PathRefusedException($"'{segment}' is not a valid name.");
    }
}
string joined = System.IO.Path.GetFullPath(System.IO.Path.Combine(Root,
    System.IO.Path.Combine(segments)));
if (!IsInside(joined))
{
    throw new
        PathRefusedException("A path must stay inside the home
        directory.");
}
return joined;
```

The tests that hold it

Every refusal is a case, and the prefix case is the one most guards miss: /home/files-old is not inside /home/files.

```
tests/TestProject.Tests/HomePathTests.cs
[Theory]
[InlineData("../secrets")]
[InlineData("docs/../../secrets")]
[InlineData("docs/./notes")]
[InlineData("../")]
public void A_dot_or_dot_dot_segment_is_refused(string path)
{
    var refused = Assert.Throws<PathRefusedException>(() =>
        _home.Resolve(path));
    Assert.Contains("'.' or '..'", refused.Message,
        StringComparison.Ordinal);
}

public void The_root_with_a_prefix_in_common_is_outside()
{
    // /home/files-old starts with the same characters as /home/files
    // and is not inside it.
    Assert.False(_home.IsInside(_home.Root + "-old"));
    Assert.False(_home.IsInside(Path.Combine(_home.Root + "-old",
        "x")));
}
```

Every failure is a sentence (ADR-004)

One handler maps each refusal to a status and writes an RFC 9457 problem document with a trace id. The controllers have no try/catch.

```
Controllers/BrowserProblemHandler.cs
(int status, string title) = exception switch
{
    BrowserProblemException problem => (problem.Status, problem.Title),
    PathRefusedException => (StatusCodes.Status400BadRequest,
        "The path was refused"),
    UnauthorizedAccessException => (StatusCodes.Status403Forbidden,
        "The filesystem refused"),
    IOException => (StatusCodes.Status409Conflict,
        "The filesystem refused"),
    _ => (0, string.Empty),
};
...
Detail = exception.Message,
// The trace id the logs carry, so a person quoting a failure can be
// found in them.
Extensions = { ["traceId"] = Activity.Current?.Id ??
    HttpContext.TraceIdentifier },
```

A search that stops (ADR-008)

The walk is lazy and the cap is clamped, so a search that fills up in the first subfolder never reads the rest of the tree, and the reply says truncated.

```
Application/FileBrowser.cs, Search
int cap = Math.Clamp(limit ?? options.SearchLimit, 1,
    options.SearchCeiling);
string absolute = Folder(path);
var folders = new List<FolderEntry>();
var files = new List<FileEntry>();
bool truncated = false;
foreach (StoreEntry entry in store.Descendants(absolute))
{
    if (!pattern.Matches(Path.GetFileName(entry.Absolute)))
    {
        continue;
    }
    if (folders.Count + files.Count >= cap)
    {
        truncated = true;
        break;
    }
    Sort(entry, folders, files);
}
folders.Sort((a, b) => ViewTotals.NameOrder.Compare(a.Path, b.Path));
files.Sort((a, b) => ViewTotals.NameOrder.Compare(a.Path, b.Path));
return new SearchResult(pattern.Query, Home.Relative(absolute), folders,
    files, ViewTotals.Of(folders, files), truncated, Elapsed(started));
```

The address is the state (ADR-005)

One parser and one serializer. A value outside its list falls back, a default is never written, so a link is as short as it can be.

```
src/lib/urlState.ts
export function parse(search: string): State {
    const params = new URLSearchParams(search.startsWith('?') ?
        search.slice(1) : search);
    return {
        // Any known key opens the dialog, so "?path=docs" is enough for a
        // link and "?view=browse" is
        // what a bare open serializes to (ADR-007).
        open: KEYS.some((key) => params.has(key)),
        view: pick(params.get('view'), VIEWS, DEFAULTS.view),
        path: cleanPath(params.get('path')),
        q: (params.get('q') ?? '').trim(),
        sort: pick(params.get('sort'), SORTS, DEFAULTS.sort),
        dir: pick(params.get('dir'), DIRS, DEFAULTS.dir),
        doc: (params.get('doc') ?? '').trim().toLowerCase(),
    };
}

export function serialize(state: Partial<State>): string {
    const full: State = { ...DEFAULTS, open: true, ...state, path:
        cleanPath(state.path) };
    if (!full.open) {
        return '';
    }
    const params = new URLSearchParams();
    for (const key of KEYS) {
        if (full[key] !== DEFAULTS[key] && full[key] !== '') {
            params.set(key, full[key]);
        }
    }
    if ([...params.keys()].length) {
        params.set('view', full.view);
    }
    return `?${params.toString}`;
}
```

The one way the state changes (ADR-005)

Merge, write the address, redraw. Closing is navigating to the defaults, so Back from a closed page reopens it where it was.

```
src/main.ts, navigate
function navigate(partial: Partial<State>, replaceEntry = false): void {
    const next: State = { ...state, open: true, ...partial };
    // A closed state serializes to "", and pushState needs an address, so
    // the bare path stands in.
    const url = serialize(next) || window.location.pathname;
    if (replaceEntry) {
        window.history.replaceState(null, '', url);
    } else {
        window.history.pushState(null, '', url);
    }
    // Re-read from the address rather than trusting `next`: the browser
    // is the one source, and a
    // value the serializer dropped (a default) must not survive in memory
    // either.
    state = parse(window.location.search);
    render();
}
```